Sistemas Operacionais

Algoritmos de escalonamento II – Sincronização de Processos e Deadlocks

Rafael Descio, M.Sc.

Católica SC
Centro Universitário

Objetivos da Aula de Hoje

- **Compreender** o conceito de escalonamento de CPU, identificando seus objetivos principais, como maximização de throughput, redução do tempo de resposta e garantia de uso eficiente da CPU.
- **Analisar** os principais algoritmos de escalonamento (FCFS, SJF, Round Robin e Prioridades), comparando seu comportamento, vantagens, limitações e impactos nas métricas de desempenho dos processos.
- **Entender** os desafios da concorrência em sistemas operacionais, reconhecendo problemas como condições de corrida e a necessidade de mecanismos de sincronização, como locks e semáforos, para controle de acesso a recursos compartilhados.
- **Compreender** o conceito de deadlock, identificando suas condições necessárias, analisando cenários clássicos de bloqueio e avaliando estratégias de prevenção, detecção e recuperação em sistemas concorrentes.
- **Aplicar** conceitos de escalonamento e concorrência por meio de exercícios práticos, incluindo simulação manual de algoritmos, cálculo de métricas (tempo de espera e resposta) e análise de situações reais envolvendo sincronização e deadlock.

Escalonamento de CPU: Revisão

O escalonamento de CPU é o mecanismo pelo qual o sistema operacional seleciona, entre os processos ou threads prontos, qual será executado pela CPU em um determinado instante.

Por que ele existe?

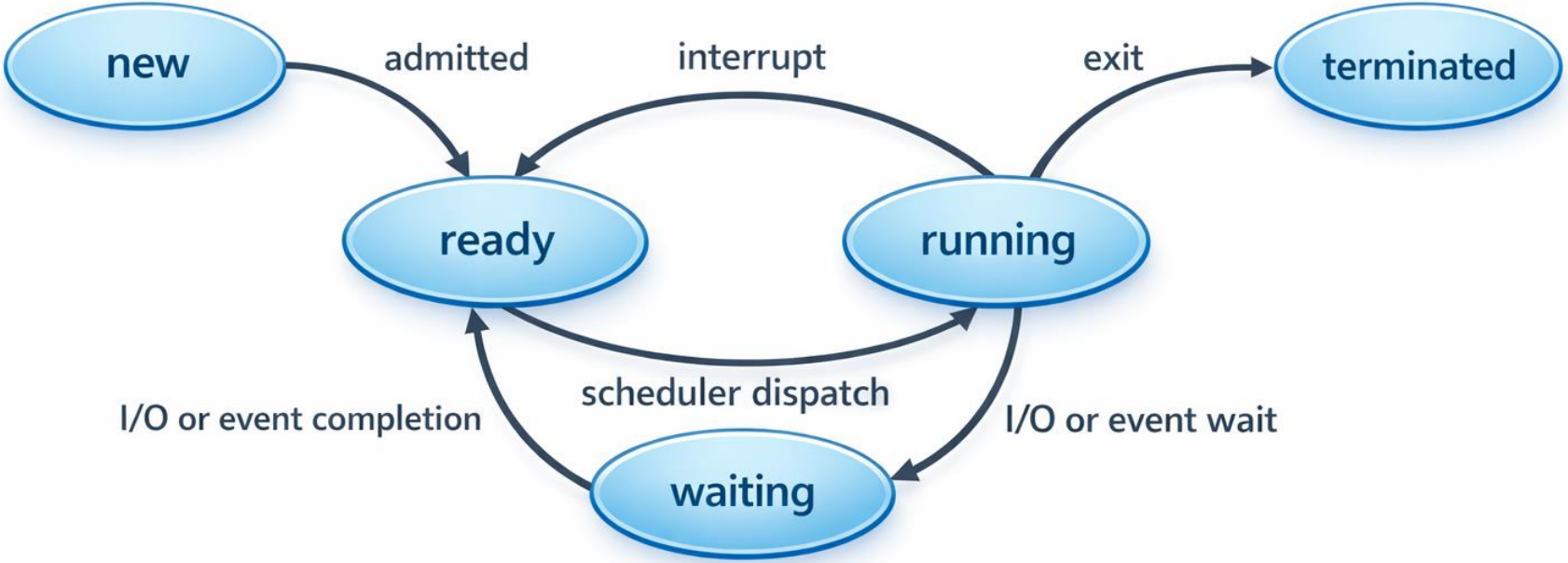
Devido à limitação de recursos computacionais, múltiplos processos competem pela CPU, exigindo uma estratégia eficiente de alocação.

Objetivos do escalonamento:

- Maximizar o throughput (processos concluídos por unidade de tempo)
- Minimizar o tempo de resposta (tempo até a primeira execução)
- Minimizar o turnaround time (tempo total de execução do processo)
- Garantir fairness (equidade no uso da CPU entre processos)

Para pensar (Tanenbaum): Não existe algoritmo ideal, cada estratégia otimiza diferentes métricas, implicando trade-offs.

Exemplo: Gerenciamento do CPU



Métricas de Desempenho em Escalonamento

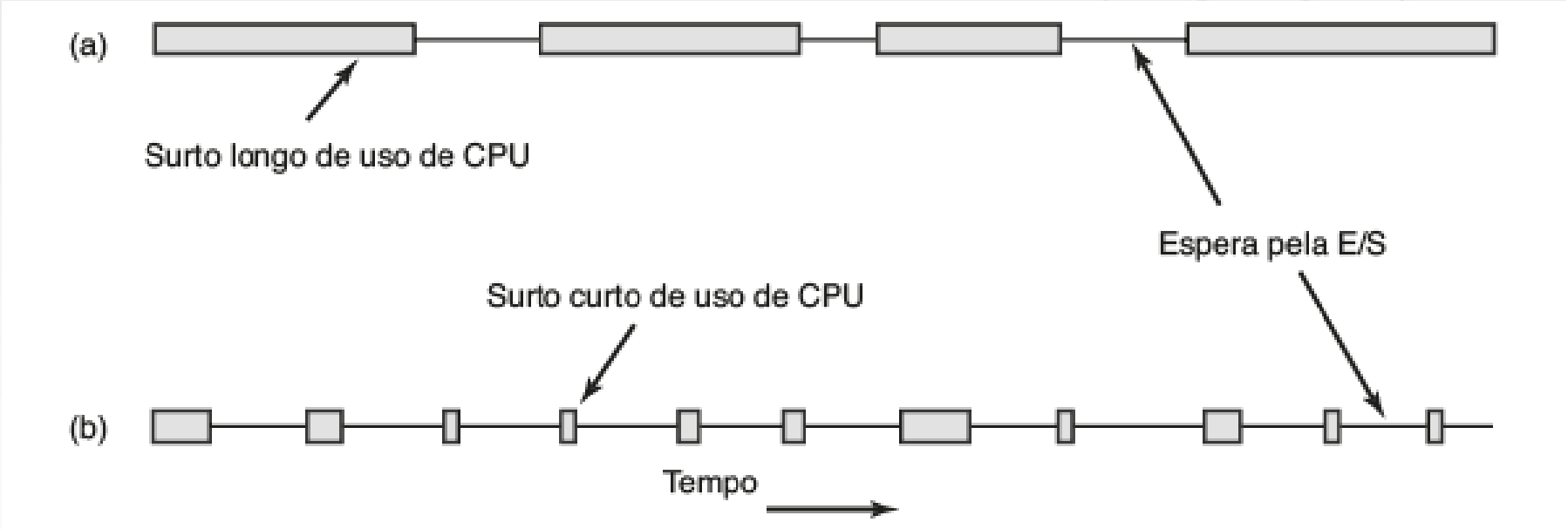
Principais métricas utilizadas:

- Tempo de espera (Waiting Time): Tempo total que o processo permanece na fila de prontos aguardando execução.
- Tempo de resposta (Response Time): Intervalo entre a chegada do processo e o início de sua execução.
- Tempo de turnaround (Turnaround Time): Tempo total desde a submissão até a conclusão do processo.
- Utilização da CPU: Percentual de tempo em que a CPU está efetivamente executando processos.

Relação entre métricas:

- Reduzir tempo de resposta melhora interatividade
- Reduzir turnaround melhora eficiência global
- Alta utilização da CPU pode aumentar tempo de espera

Exemplos:


(a)
Surto longo de uso de CPU
Espera pela E/S
Surto curto de uso de CPU
(b)
Tempo

Algoritmos Clássicos

Principais algoritmos:

- FCFS (problema do convoy effect)
- SJF (ótimo teórico + inviabilidade prática)
- SRTF (preempção agressiva + risco de starvation)
- Round Robin (quantum trade-off)
- Prioridades (starvation e aging)
- Multilevel Queue (rigidez entre filas)
- Multilevel Feedback Queue (adaptação dinâmica + maior complexidade)

Algoritmos Clássicos

FCFS (First-Come, First-Served)

Tipo: Não-preemptivo

Critério: Ordem de chegada

Vantagens:
- Simples implementação
- Baixo overhead

Desvantagens:
- Convoy effect (processos curtos ficam atrás de longos)
- Alto tempo médio de espera

Quando usar:
- Sistemas batch simples

SJF (Shortest Job First)

Tipo: Não-preemptivo (versão preemptiva: SRTF)

Critério: Menor tempo de execução (burst)

Vantagens:
- Minimiza tempo médio de espera (ótimo teórico)

Desvantagens:
- Necessita estimativa do tempo de execução
- Pode causar starvation

Quando usar:
- Ambientes previsíveis (batch controlado)

Algoritmos Clássicos

SRTF (Shortest Remaining Time First)

Tipo: Preemptivo

Critério: Menor tempo restante

Vantagens:
- Melhor tempo médio de resposta

Desvantagens:
- Alto overhead de preempção
- Starvation de processo longos

Quando usar:
- Sistemas com alta variação de duração

Round Robin (RR)

Tipo: Preemptivo

Critério: Quantum de tempo fixo

Vantagens:
- Boa responsividade
- Fairness entre processos

Desvantagens:
- Overhead alto se quanto pequeno
- Ineficiência se quantum grande

Quando usar:
- Sistemas interativos

Algoritmos Clássicos

Escalonamento por Prioridades

Tipo: Preemptivo ou não

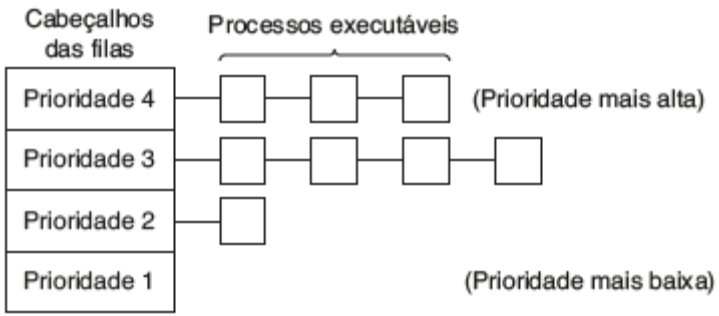
Critério: Nível de prioridade

Vantagens:
- Permite diferencia importância dos processos

Desvantagens:
- Starvation de processo de baixa prioridade

Solução clássica:
- Aging (aumento gradual da prioridade)

Quando usar:
- Sistema com criticidade diferenciada



Algoritmos Clássicos

Multilevel Queue (MLQ)

Tipo: Múltiplas filas fixas

Critério: Classe de processo (ex: sistema, interativo, batch)

Vantagens:
- Isolamento entre tipos de workload

Desvantagens:
- Rigidez (processo não muda na fila)

Quando usar:
- Sistemas com perfil bem definido

Multilevel Feedback Queue (MLFQ)

Tipo: Dinâmico e adaptativo

Critério: Comportamento do processo (histórico de uso da CPU)

Vantagens:
- Equilibra throughput e responsividade
- Reduz starvation

Desvantagens:
- Complexidade maior

Quando usar:
- Sistemas modernos (ex: Linux, Windows)

Avaliação de Algoritmos de Escalonamento

O desempenho de um algoritmo de escalonamento **não é absoluto**, sendo diretamente dependente da **carga de trabalho (workload)** e do **contexto** do sistema operacional.

1. Dependência da Carga de Trabalho (Workload)

O comportamento do algoritmo varia conforme características dos processos:

- CPU-bound vs I/O-bound
  - CPU-bound → processos longos favorecem algoritmos como SJF
  - I/O-bound → processos curtos favorecem resposta rápida (RR)
- Tempo de execução (burst time)
  - Alta variabilidade → pode penalizar FCFS (convoy effect)
  - Previsibilidade → favorece SJF
- Taxa de chegada de processos
  - Alta concorrência → exige preempção (ex: Round Robin)
  - Baixa concorrência → algoritmos simples podem ser suficientes

Avaliação de Algoritmos de Escalonamento

2. Dependência do Contexto do Sistema

O ambiente onde o algoritmo opera altera completamente seu desempenho:

- Sistemas interativos (ex: desktop, mobile)
  - Prioridade: baixo tempo de resposta
  - Algoritmos típicos: Round Robin + prioridades
- Sistemas batch (ex: processamento em lote)
  - Prioridade: throughput e turnaround
  - Algoritmos típicos: SJF, FCFS
- Sistemas em tempo real
  - Prioridade: previsibilidade e deadlines
  - Algoritmos específicos (ex: EDF, Rate Monotonic)

Avaliação de Algoritmos de Escalonamento

Trade-offs inevitáveis

Melhorar uma métrica geralmente degrada outra:

- ↓ Tempo de resposta → ↑ Troca de contexto (overhead)
- ↑ Throughput → pode ↑ tempo de espera
- ↑ Fairness → pode reduzir eficiência global

Implicação prática (segundo Tanenbaum)

Não existe algoritmo universalmente ótimo

A escolha deve considerar:

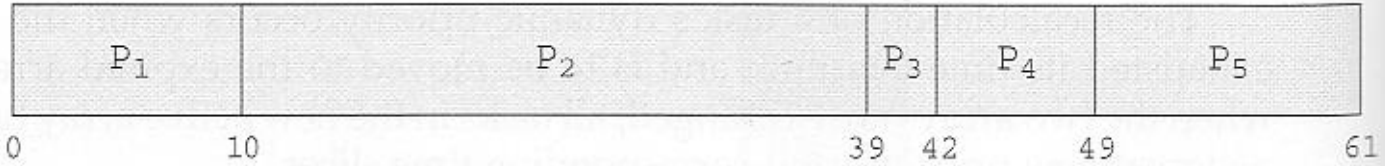
- Tipo de aplicação
- Perfil de carga
- Objetivo do sistema

Workload × Algoritmos de Escalonamento

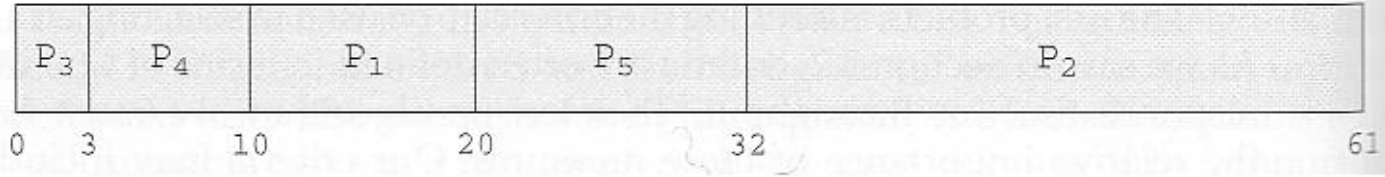
Tipo de Workload	Características	Algoritmos Mais Adequados	Justificativa Técnica
CPU-bound	Processos longos, pouca interação com I/O	SJF / Prioridades	Minimiza tempo médio de execução e melhora throughput
I/O-bound	Processos curtos, frequentes pausas para I/O	Round Robin	Garante rápida resposta e evita starvation
Interativo (tempo real leve)	Usuário espera resposta imediata (ex: UI, apps)	Round Robin + Prioridades	Baixo tempo de resposta e sensação de fluidez
Batch (lote)	Processos longos, sem interação com usuário	FCFS / SJF	Maximiza throughput e eficiência global
Alta concorrência	Muitos processos competindo simultaneamente	Round Robin	Distribuição justa da CPU (fairness)
Tempo real (crítico)	Requisitos de deadline rígidos	Escalonamento por prioridades (RT)	Garante previsibilidade e cumprimento de prazos

Exemplos:

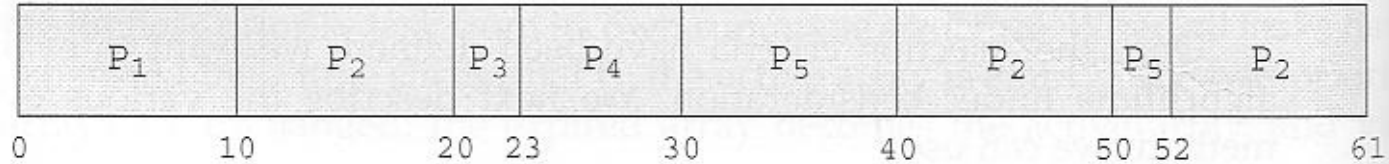
FCFS:



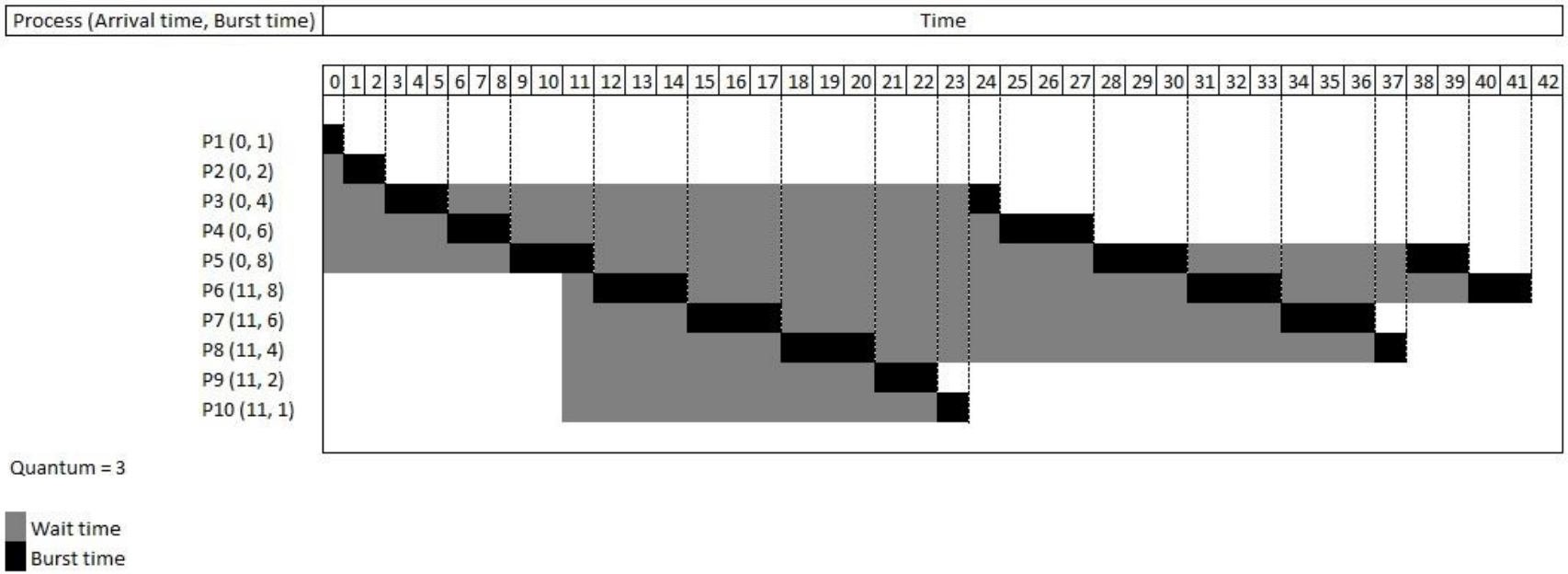
Non-preemptive SJF:



Round Robin:



Exemplos - RR:


Process (Arrival time, Burst time)
Time
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42
P1 (0, 1)
P2 (0, 2)
P3 (0, 4)
P4 (0, 6)
P5 (0, 8)
P6 (11, 8)
P7 (11, 6)
P8 (11, 4)
P9 (11, 2)
P10 (11, 1)
Quantum = 3
Wait time
Burst time

Sincronização

- Problema da região crítica
- Race condition
- Mutex vs Semáforo
- Problema clássico: Produtor-Consumidor

Mutex e Locking

Mutex (Mutual Exclusion)

- Mecanismo que garante que apenas um processo/thread por vez acesse um recurso crítico.

Locking

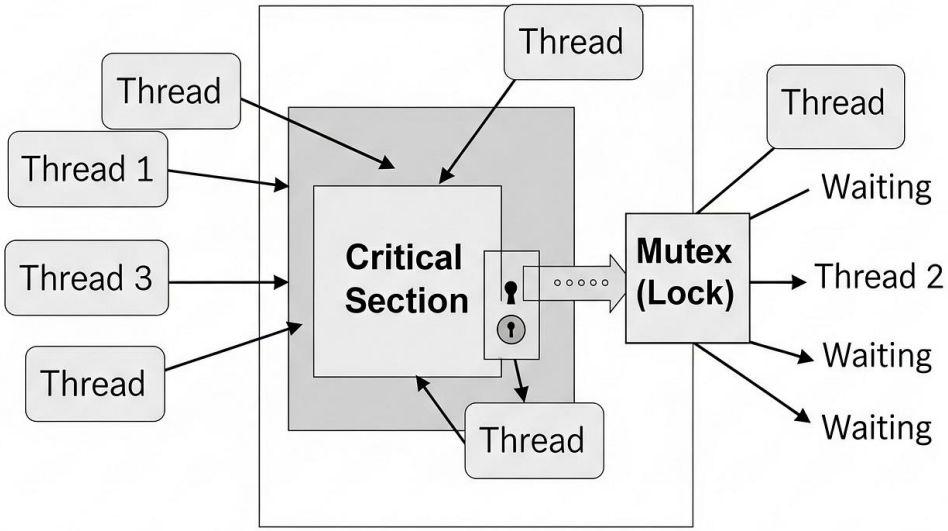
- Estratégia geral de controle de acesso usando locks (travas). Mutex é um tipo de lock.

Mutex e Locking

Mutex = primitiva de baixo nível

Base para:

- Semáforos
- Monitores
- Condições



Região Crítica

Trecho do código onde:

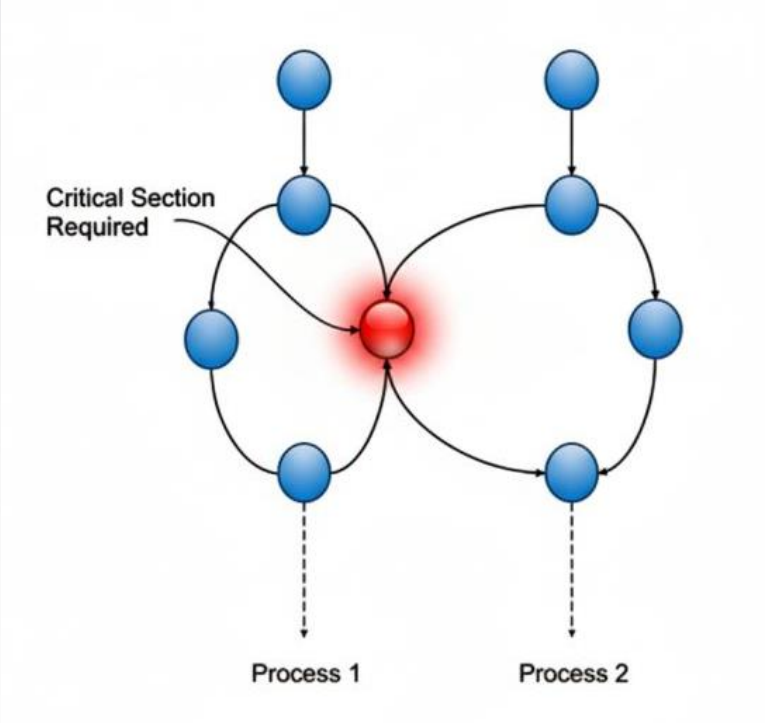
- Há acesso a recurso compartilhado
- Risco de race condition

lock() → entra na região crítica
unlock() → libera o acesso

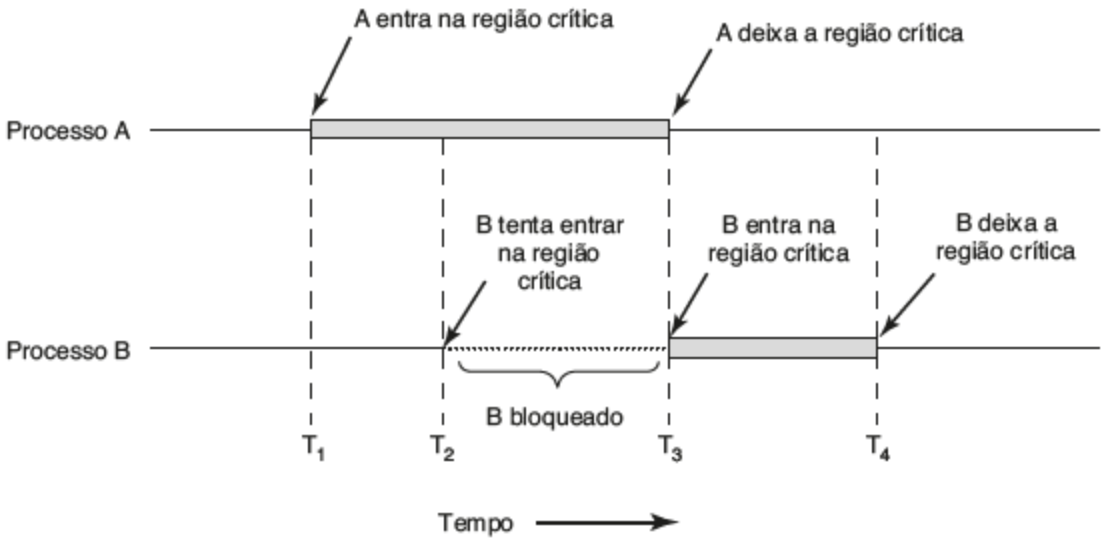
Cuidados com Deadlock:

Thread 1:	Thread 2:
lock(A)	lock(B)
lock(B)	lock(A)

Trava o Sistema!



Região Crítica



Produtor - Consumidor

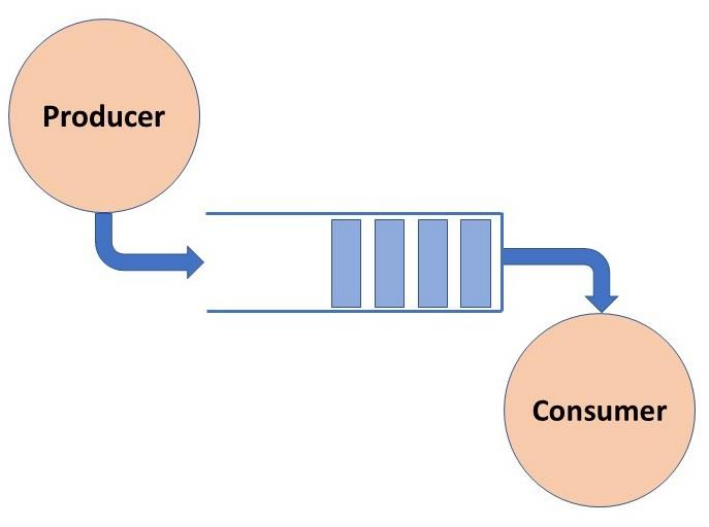
O problema de Produtor e Consumidor modela duas atividades concorrentes:

- Produtor: gera itens
- Consumidor: retira e usa itens

Ambos compartilham um buffer finito.

O problema produtor-consumidor é um modelo clássico de sincronização. Ao mesmo tempo temos:

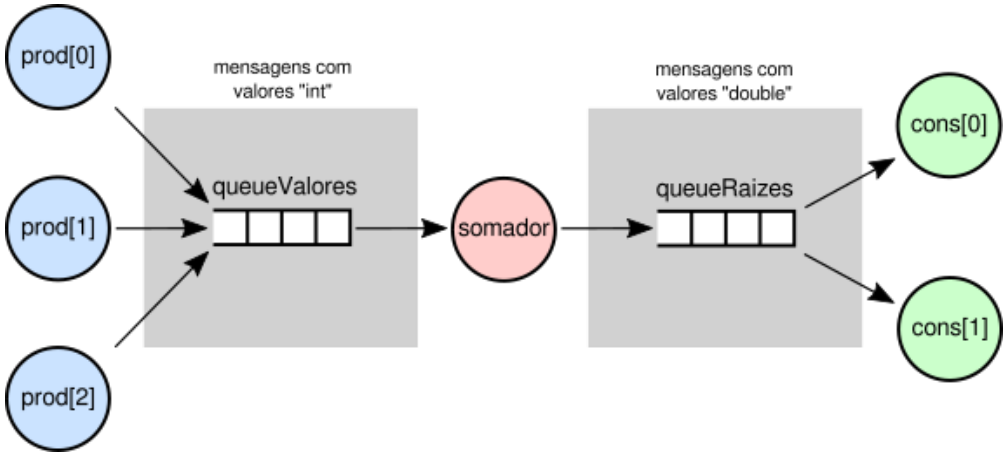
- Concorrência
- compartilhamento de recursos
- exclusão mútua
- espera controlada
- coordenação entre processos



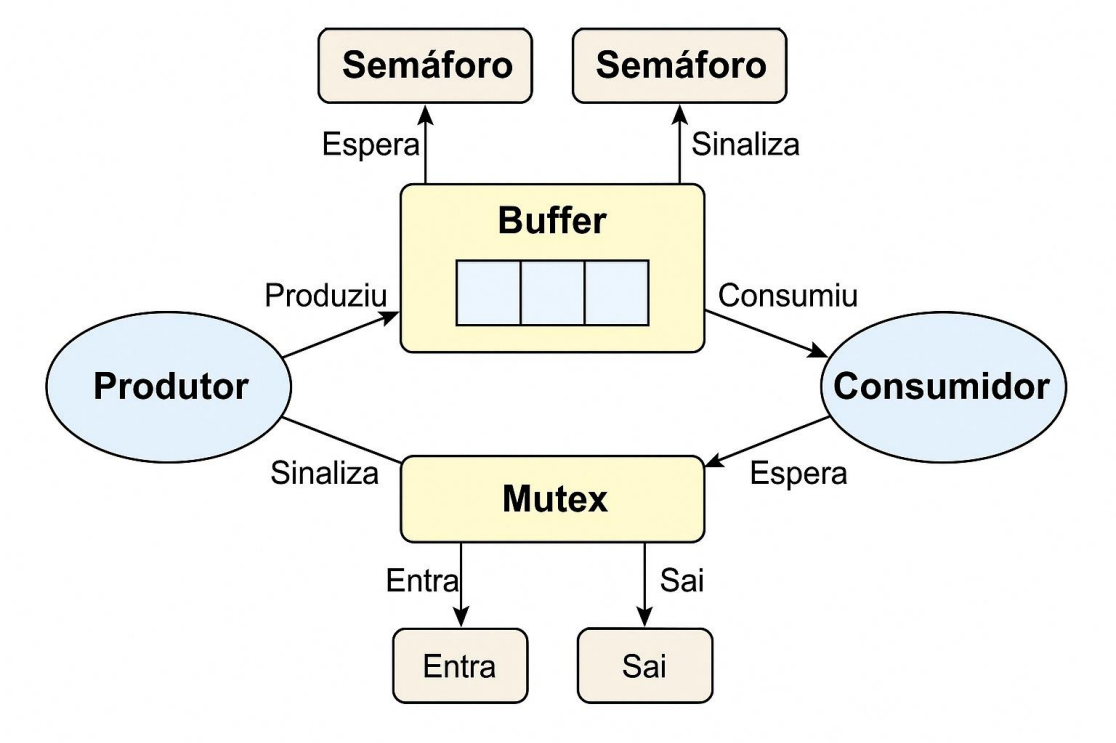
Produtor - Consumidor

Esse problema aparece em vários contextos reais:

- spool de impressão
- filas de mensagens
- pipelines de processamento
- streaming de dados
- filas de requisições em servidores
- troca de dados entre threads



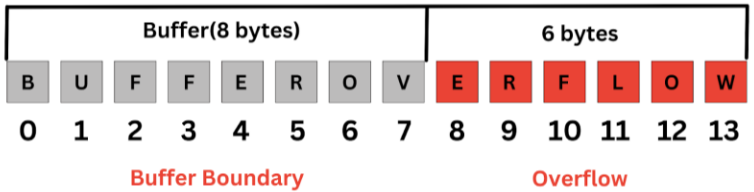
Sincronização – Produtor - Consumidor



Definição - Buffer

Buffer é uma área de memória temporária usada para armazenar dados enquanto eles estão sendo transferidos entre dois componentes ou processos que operam em velocidades diferentes.

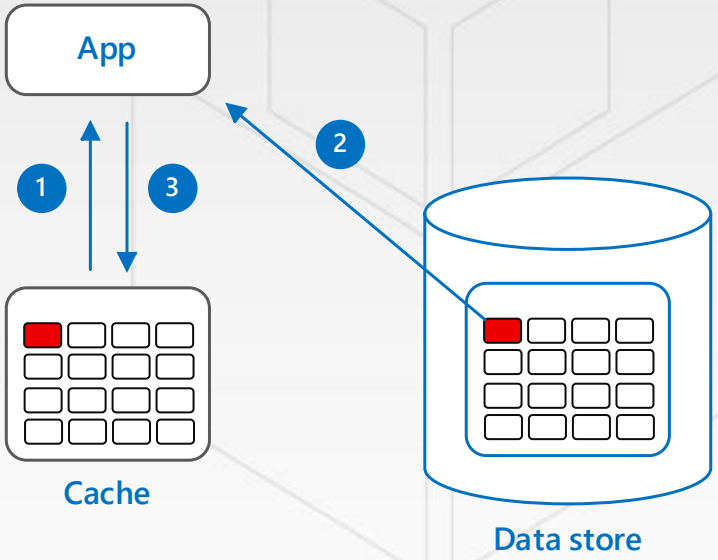
- Serve para amortecer diferenças de velocidade entre produtor e consumidor de dados;
- Evita perda de dados ou ociosidade desnecessária;
- É muito usado em entrada/saída (I/O), comunicação entre processos, disco, rede, teclado, impressora e pipelines.



Definição - Cache

É uma memória pequena e rápida que guarda cópias de dados usados com frequência, para acelerar acessos futuros.

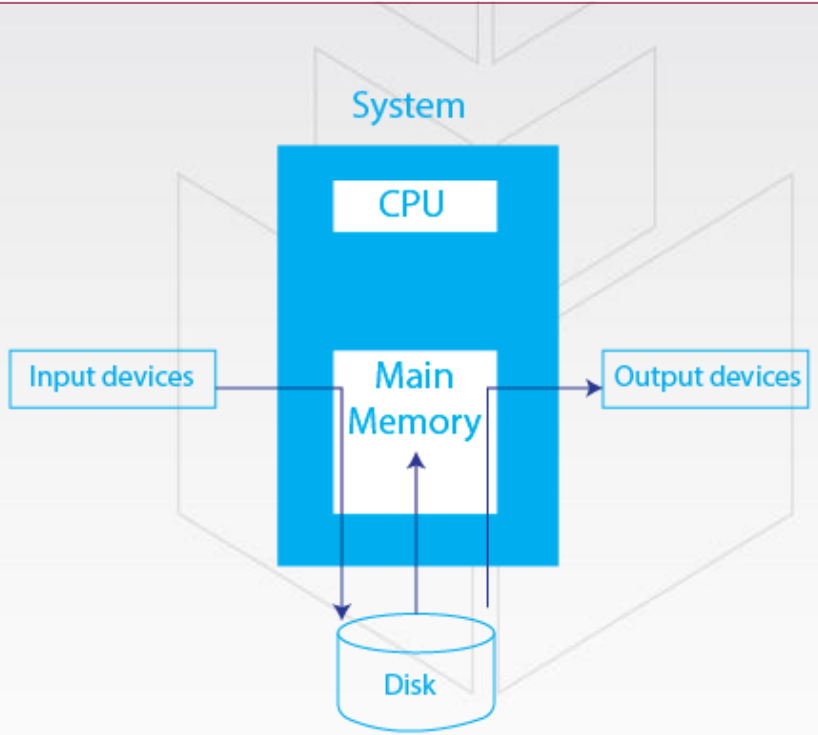
A cache existe porque acessar certos recursos é mais lento do que acessar uma memória rápida intermediária. Então, o sistema guarda ali os dados mais prováveis de serem reutilizados.



Definição – Spool (spooling) (Simultaneous Peripheral Operation On-Line)

Spooling é uma técnica em que os dados ou trabalhos destinados a um dispositivo são armazenados temporariamente em uma área intermediária, normalmente em disco, para serem processados depois, em sequência.

O spool é usado quando vários processos querem utilizar um mesmo dispositivo, especialmente um dispositivo lento ou compartilhado.

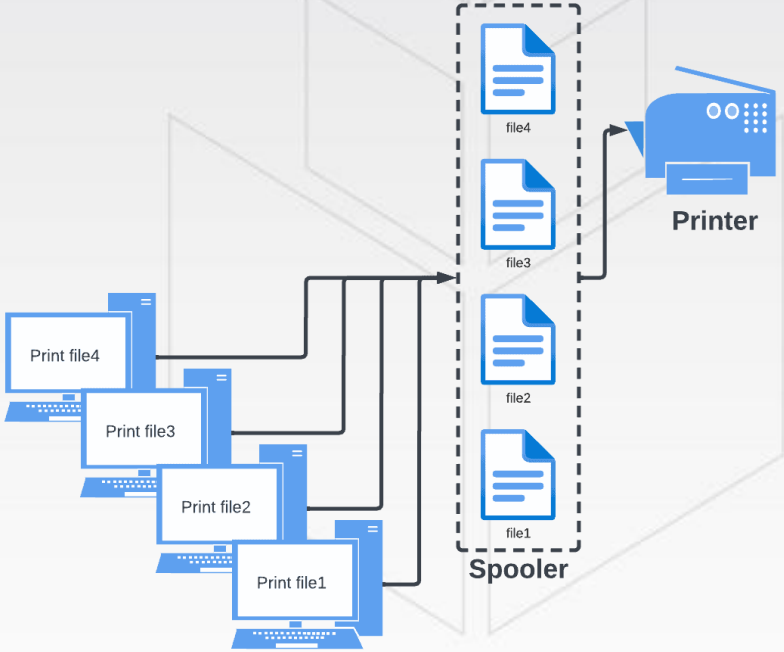


Definição – Spool (spooling) (Simultaneous Peripheral Operation On-Line)

Exemplo clássico

A impressora:

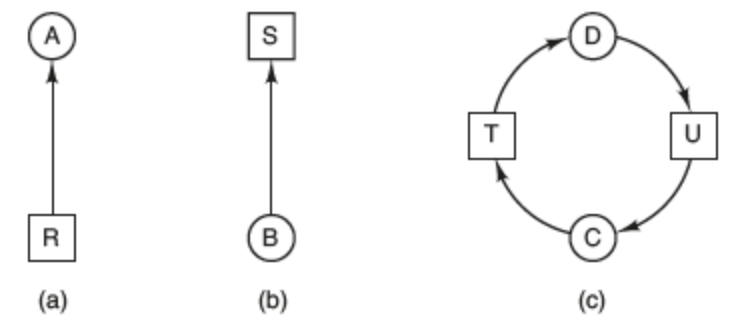
- Vários usuários enviam documentos ao mesmo tempo;
- Os trabalhos são colocados em uma fila;
- O sistema os libera para a impressora um por vez.



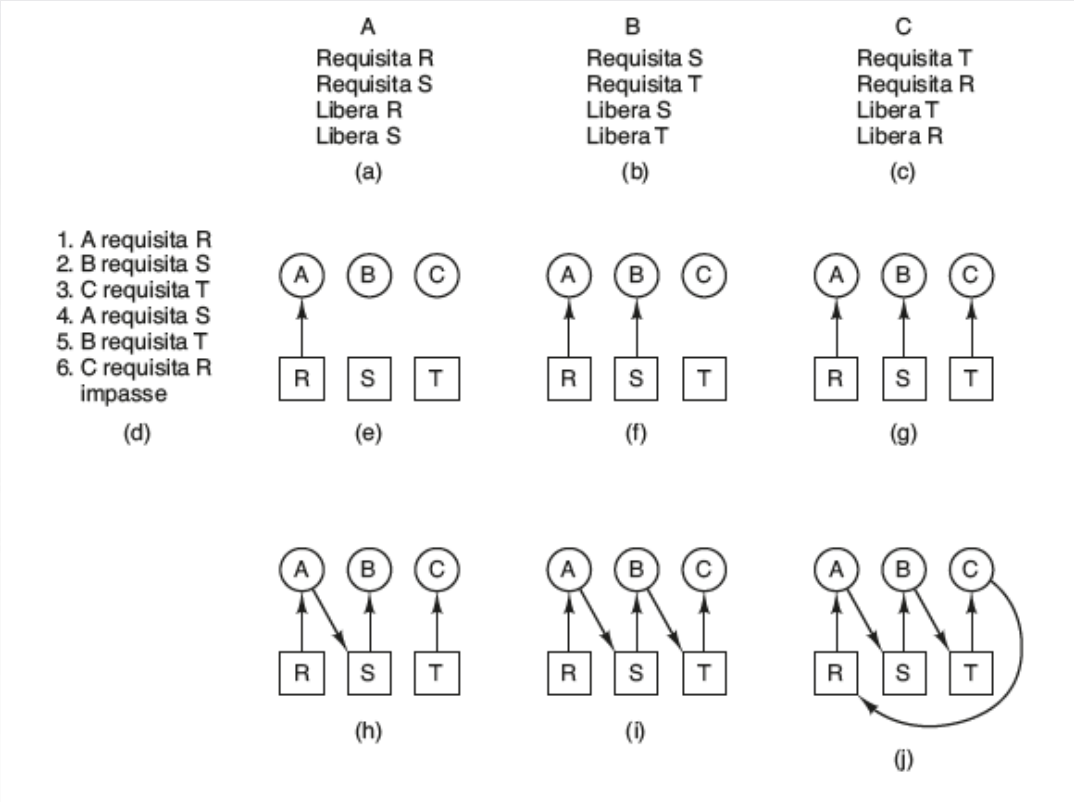
Impasses

Coffman et al. (1971) demonstraram que quatro condições têm de ser válidas para haver um impasse (de recurso):

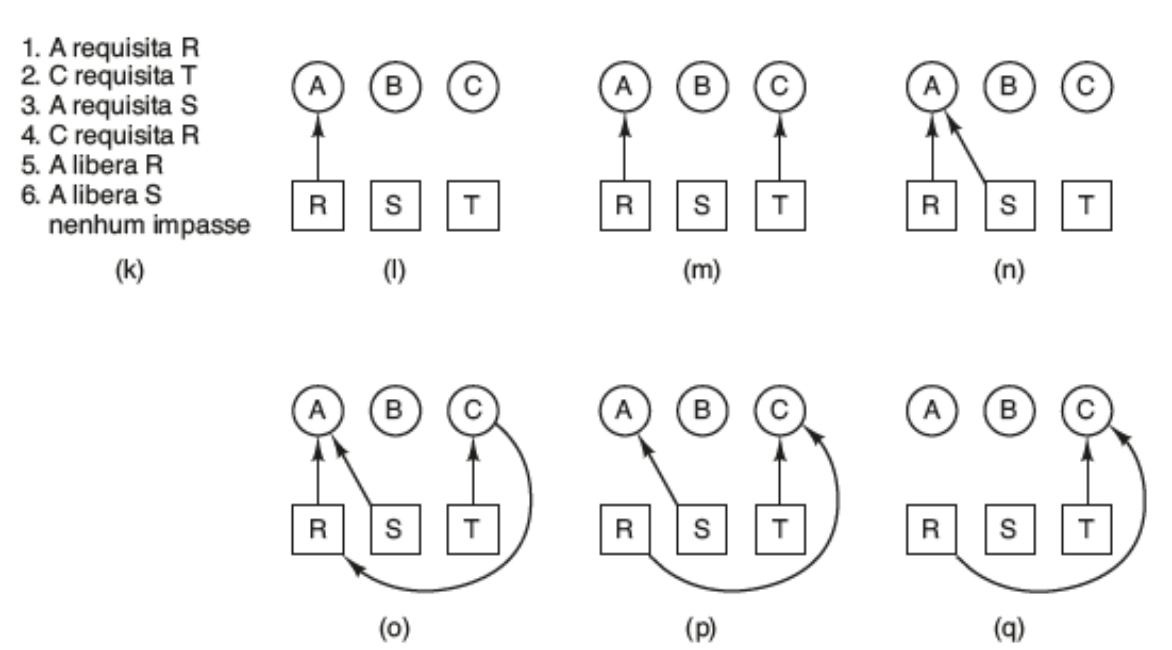
1. Condição de exclusão mútua. Cada recurso está atualmente associado a exatamente um processo ou está disponível.
2. Condição de posse e espera. Processos atualmente de posse de recursos que foram concedidos antes podem solicitar novos recursos.
3. Condição de não preempção. Recursos concedidos antes não podem ser tomados à força de um processo. Eles precisam ser explicitamente liberados pelo processo que os têm.
4. Condição de espera circular. Deve haver uma lista circular de dois ou mais processos, cada um deles esperando por um processo



Impasses


A
Requisita R
Requisita S
Libera R
Libera S
(a)
B
Requisita S
Requisita T
Libera S
Libera T
(b)
C
Requisita T
Requisita R
Libera T
Libera R
(c)
1. A requisita R
2. B requisita S
3. C requisita T
4. A requisita S
5. B requisita T
6. C requisita R
impasse
(d)
(e)
(f)
(g)
(h)
(i)
(j)

Impasses


1. A requisita R
2. C requisita T
3. A requisita S
4. C requisita R
5. A libera R
6. A libera S
nenhum impasse
(k)
A B C
R S T
(l)
(m)
(n)
(o)
(p)
(q)

Impasses = Deadlock

4 condições necessárias para que ocorra:

- Exclusão mútua
- Hold and wait
- No preemption
- Espera circular

Exemplos:

- Jantar dos filósofos
- Resource allocation graph

Deadlock

Estratégias:

- Prevenção
- Evitação (Banker’s Algorithm — conceitual)
- Detecção e recuperação

Impasse - Prevenção

Evitar que o deadlock possa acontecer.

Como fazer:

Eliminar uma das 4 condições necessárias

- Exclusão mútua
- Hold and wait
- Não-preempção
- Espera circular

Ordenar recursos (ex: sempre pegar R1 → R2 → R3)

Solicitar todos os recursos de uma vez

Condição	Abordagem contra impasses
Exclusão mútua	Usar spool em tudo
Posse e espera	Requisitar todos os recursos necessários no início
Não preempção	Retomar os recursos alocados
Espera circular	Ordenar numericamente os recursos

Trade-off:

reduz risco, mas diminui paralelismo

Impasse - Evitação (Banker's Algorithm — conceitual)

Só permitir execuções seguras

- Conceito-chave:
  Estado seguro → existe uma ordem em que todos terminam
- Estado inseguro → pode levar a deadlock

Como funciona:

- Simula alocação antes de conceder recurso
- Só libera se o sistema continuar seguro

Trade-off:

mais inteligente, porém alto custo computacional

Impasse - Detecção e Recuperação

Deixar acontecer e depois tratar

Detecção:

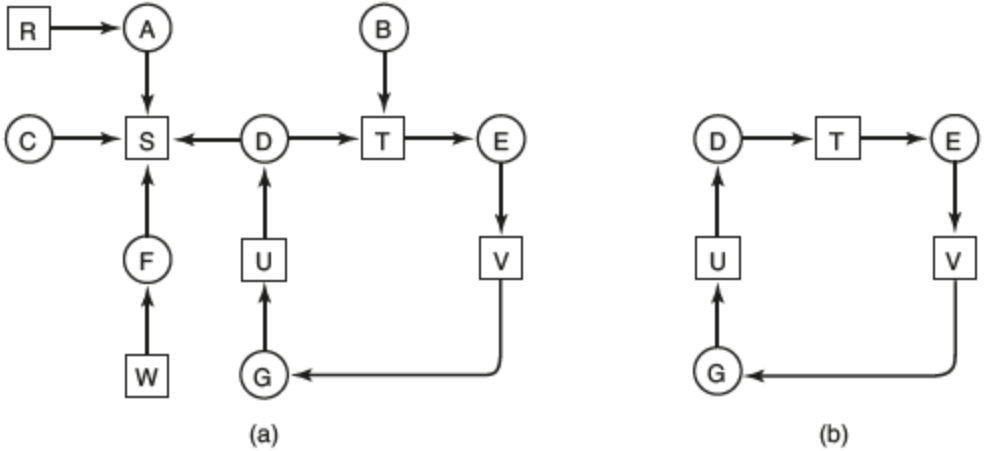
- Grafo de espera (wait-for graph)
- Identificação de ciclos

Recuperação:

- Matar processos
- Rollback (voltar estado)
- Forçar liberação de recursos

Trade-off:

simples de implementar, mas pode ter impacto operacional alto



Exemplo Simples - Passo a passo

João (P1) pega a caneta (R1)

Maria (P2) pega o caderno (R2)

Exemplo Simples - Passo a passo

João precisa do caderno

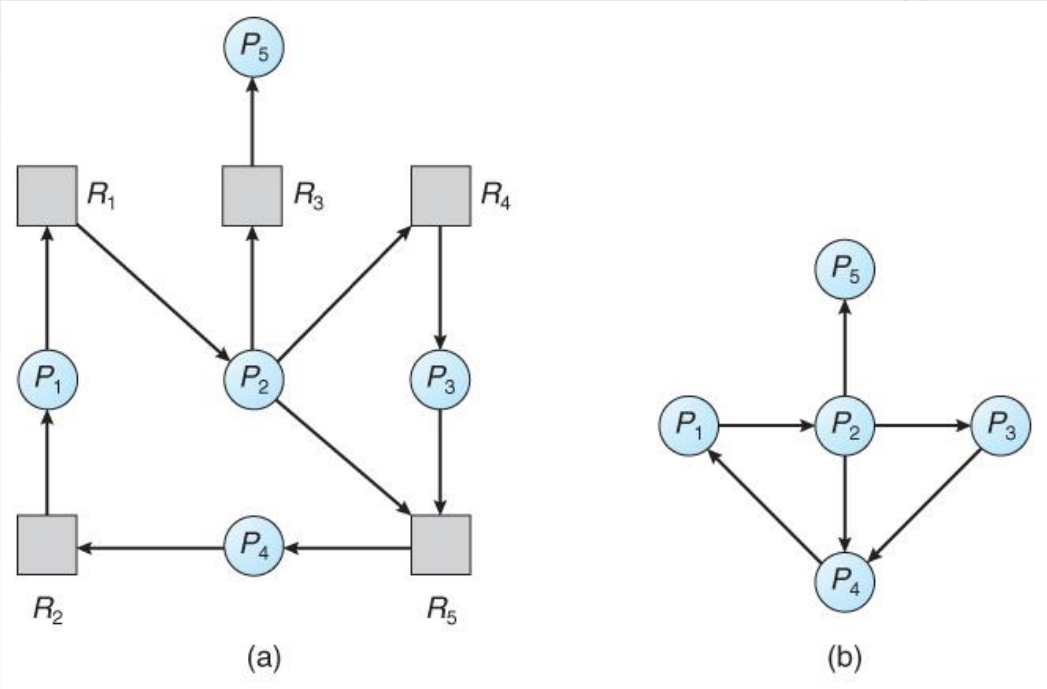
Maria precisa da caneta

- P1 → R2
- P2 → R1

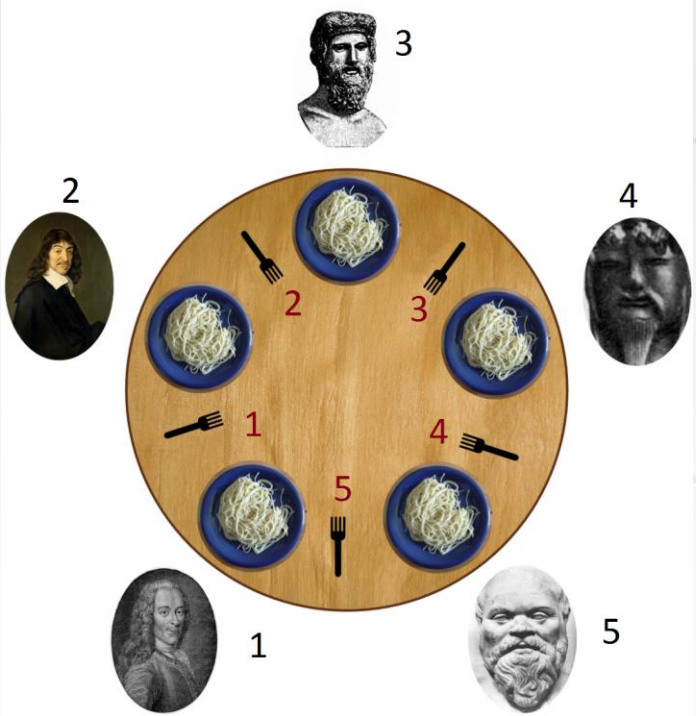
P1 → R2 → P2 → R1 → P1

- João não solta a caneta
- Maria não solta o caderno
- Ambos ficam esperando

Resource Allocation Graph – Process and Resources



Jantar dos filósofos



Entendendo o problema do
Jantar dos Filósofos!
MINECRAFT TECH
PROFESSOR

Semáforos (Semaphore)

Mecanismo criado pelo matemático holandês E.W. Dijkstra, em 1965.

O semáforo é uma variável inteira que pode ser alterada por apenas duas operações primitivas (atômicas): P e V.

- P = proberen (testar)
- V = verhogen (incrementar).

Quando um processo executa uma operação P, o valor do semáforo é decrementado (se o semáforo for maior que 0).

O processo pode ser eventualmente bloqueado (semáforo for igual a 0) e inserido na fila de espera do semáforo.

Numa operação V, o semáforo é incrementado e, eventualmente, um processo que aguarda na fila de espera deste semáforo é acordado.

Semáforos (Semaphore)

Operação P também é comumente referenciada como:

- *down* ou *wait*

Operação V também é comumente referenciada como:

- *up* ou *signal*

Semáforos que assumem apenas os valores 0 e 1 são denominados **semáforos binários** ou **mutex**. Neste caso, P e V são também chamadas de LOCK e UNLOCK, respectivamente.

Monitores (Monitor)

Sincronização de alto nível para acesso seguro a recursos compartilhados

Definição:

Monitor é uma estrutura de sincronização que encapsula:

- dados compartilhados
- operações sobre esses dados
- exclusão mútua automática
- mecanismos de espera e sinalização

Quando uma thread entra no monitor, nenhuma outra pode executar nele ao mesmo tempo.

Monitores (Monitor)

Elementos principais

- Procedimentos do monitor: funções que acessam o recurso compartilhado
- Variáveis de condição: usadas para esperar e sinalizar eventos
- Exclusão mútua implícita: controlada pelo próprio monitor

Operações típicas:

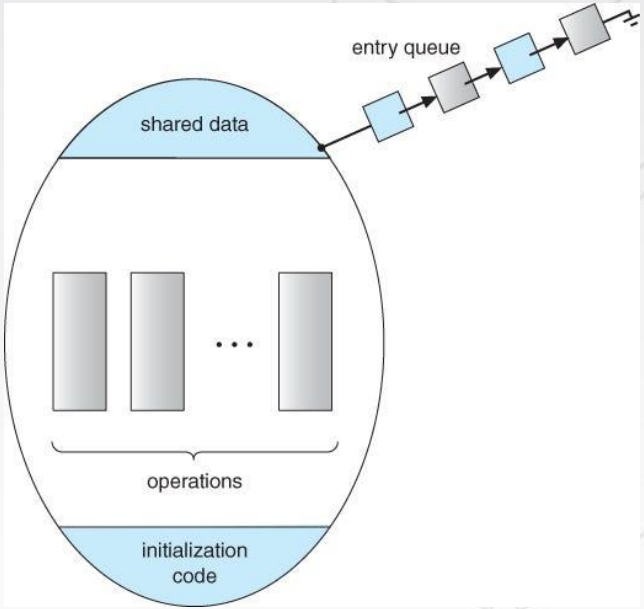
- wait() → bloqueia a thread até que uma condição seja satisfeita
- signal() → acorda uma thread que estava esperando

Monitores (Monitor)

Vantagens

- código mais organizado
- menor chance de erro do que semáforos
- abstração mais segura para sincronização

Monitores combinam **exclusão mútua + espera por condição** em uma estrutura mais segura e mais fácil de entender do que semáforos isolados.



Semáforo Binário

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int contador = 0;
pthread_mutex_t mutex;

void* incrementar(void* arg) {
    for (int i = 0; i < 1000; i++) {
        pthread_mutex_lock(&mutex);
        contador++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&t1, NULL, incrementar, NULL);
    pthread_create(&t2, NULL, incrementar, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&mutex);

    printf("Valor final do contador: %d\n", contador);
    return 0;
}
```

Semáforo com Contador (Counting Semaphore)

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
sem_t sem;

void* tarefa(void* arg) {
    int id = *(int*)arg;
    printf("Thread %d aguardando recurso...\n", id);
    sem_wait(&sem); // down / P
    printf("Thread %d entrou na região crítica.\n", id);
    sleep(2);
    printf("Thread %d saindo da região crítica.\n", id);
    sem_post(&sem); // up / V
    return NULL;
}

int main() {
    pthread_t t1, t2, t3;
    int id1 = 1, id2 = 2, id3 = 3;
    sem_init(&sem, 0, 1); // semáforo binário
    pthread_create(&t1, NULL, tarefa, &id1);
    pthread_create(&t2, NULL, tarefa, &id2);
    pthread_create(&t3, NULL, tarefa, &id3);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    sem_destroy(&sem);

    return 0;
}
```

O algoritmo dos Filósofos Comilões – Jantar dos Filósofos

- Pseudocódigo - Tanenbaum

```
#define N 5                    /* numero de filosofos */
#define LEFT (i + N - 1) % N /* numero do vizinho a esquerda de i */
#define RIGHT (i + 1) % N    /* numero do vizinho a direita de i */
#define THINKING 0           /* o filosofo esta pensando */
#define HUNGRY 1             /* o filosofo esta tentando pegar garfos */
#define EATING 2             /* o filosofo esta comendo */
typedef int semaphore;       /* semaforos sao um tipo especial de int */
int state[N];                /* arranjo para controlar o estado de cada um */
semaphore mutex = 1;         /* exclusao mutua para as regioes criticas */
semaphore s[N];              /* um semaforo por filosofo */

void philosopher(int i) /* i: o numero do filosofo, de 0 a N-1 */
{
    while (true)
    {                   /* repete para sempre */
        think();        /* o filosofo esta pensando */
        take_forks(i); /* pega dois garfos ou bloqueia */
        eat();          /* hummm, espaguete! */
        put_forks(i);   /* devolve os dois garfos a mesa */
    }
}
void take_forks(int i) /* i: o numero do filosofo, de 0 a N-1 */
{
    down(&mutex);       /* entra na regiao critica */
    state[i] = HUNGRY; /* registra que o filosofo esta faminto */
```

O algoritmo dos Filósofos Comilões

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5                          /* numero de filosofos */
#define LEFT(i) (((i) + N - 1) % N) /* numero do vizinho a esquerda de i
*/
#define RIGHT(i) (((i) + 1) % N)    /* numero do vizinho a direita de i
*/
#define THINKING 0                   /* o filosofo esta pensando */
#define HUNGRY 1                     /* o filosofo esta tentando pegar
garfos */
#define EATING 2                     /* o filosofo esta comendo */
#define MAX_MEALS 3                  /* numero maximo de refeicoes por
filosofo */
```

```
filosofo */

int state[N];                       /* arranjo para controlar o estado de
cada um */
int ids[N];                         /* ids dos filosofos */
int meals[N] = {0};                 /* contador de refeicoes por filosofo
*/

sem_t mutex;                        /* exclusao mutua para as regioes
criticas */
sem_t s[N];                         /* um semaforo por filosofo */

void think(int i);
void eat(int i);
void take_forks(int i);
void put_forks(int i);
void test(int i);
void *philosopher(void *arg);

void think(int i) /* i: o numero do filosofo, de 0 a N-1 */
{
    printf("Filosofo %d esta pensando\n", i);
    sleep(1 + rand() % 3);
}
```

```c
void think(int i) /* i: o numero do filosofo, de 0 a N-1 */
{
    printf("Filosofo %d esta pensando\n", i);
    sleep(1 + rand() % 3);
}

void eat(int i) /* i: o numero do filosofo, de 0 a N-1 */
{
    printf("Filosofo %d esta comendo\n", i);
    sleep(1 + rand() % 2);
}

void *philosopher(void *arg) /* i: o numero do filosofo, de 0 a N-1 */
{
    int i = *(int *)arg;

    while (meals[i] < MAX_MEALS)
    {                           /* repete ate atingir o limite de refeicoes
*/
        think(i);               /* o filosofo esta pensando */
        take_forks(i);          /* pega dois garfos ou bloqueia */
        eat(i);                 /* hummm, espaguete! */
        meals[i]++;             /* registra uma refeicao */
```

```c
void *philosopher(void *arg) /* i: o numero do filosofo, de 0 a N-1 */
{
    int i = *(int *)arg;

    while (meals[i] < MAX_MEALS)
    {                           /* repete ate atingir o limite de refeicoes
*/
        think(i);               /* o filosofo esta pensando */
        take_forks(i);          /* pega dois garfos ou bloqueia */
        eat(i);                 /* hummm, espaguete! */
        meals[i]++;             /* registra uma refeicao */
        put_forks(i);           /* devolve os dois garfos a mesa */
    }

    printf("Filosofo %d terminou apos %d refeicoes\n", i, meals[i]);
    return NULL;
}

void take_forks(int i) /* i: o numero do filosofo, de 0 a N-1 */
{
    sem_wait(&mutex);           /* entra na regiao critica */
    state[i] = HUNGRY;          /* registra que o filosofo esta faminto */
```

```
void take_forks(int i) /* i: o numero do filosofo, de 0 a N-1 */
{
    sem_wait(&mutex);          /* entra na regiao critica */
    state[i] = HUNGRY;         /* registra que o filosofo esta faminto */
    printf("Filosofo %d esta faminto\n", i);
    test(i);                   /* tenta pegar dois garfos */
    sem_post(&mutex);          /* sai da regiao critica */
    sem_wait(&s[i]);           /* bloqueia se os garfos nao foram pegos */
}

void put_forks(int i) /* i: o numero do filosofo, de 0 a N-1 */
{
    sem_wait(&mutex);          /* entra na regiao critica */
    state[i] = THINKING;       /* o filosofo acabou de comer */
    printf("Filosofo %d devolveu os garfos\n", i);
    test(LEFT(i));             /* ve se o vizinho da esquerda pode comer
agora */
    test(RIGHT(i));            /* ve se o vizinho da direita pode comer
agora */
    sem_post(&mutex);          /* sai da regiao critica */
}

void test(int i) /* i: o numero do filosofo, de 0 a N-1 */
```

```c
void test(int i) /* i: o numero do filosofo, de 0 a N-1 */
{
    if (state[i] == HUNGRY &&
        state[LEFT(i)] != EATING &&
        state[RIGHT(i)] != EATING)
    {
        state[i] = EATING;
        sem_post(&s[i]);
    }
}

int main(void)
{
    pthread_t threads[N];

    srand((unsigned int)getpid());

    if (sem_init(&mutex, 0, 1) != 0)
    {
        perror("erro ao inicializar mutex");
        return 1;
    }
```

```c
int main(void)
{
    pthread_t threads[N];

    srand((unsigned int)getpid());

    if (sem_init(&mutex, 0, 1) != 0)
    {
        perror("erro ao inicializar mutex");
        return 1;
    }

    for (int i = 0; i < N; i++)
    {
        state[i] = THINKING;
        ids[i] = i;

        if (sem_init(&s[i], 0, 0) != 0)
        {
            perror("erro ao inicializar semaforo");
            return 1;
        }
    }
```

```c
for (int i = 0; i < N; i++)
{
    if (pthread_create(&threads[i], NULL, philosopher, &ids[i]) != 0)
    {
        perror("erro ao criar thread");
        return 1;
    }
}

for (int i = 0; i < N; i++)
{
    pthread_join(threads[i], NULL);
}

for (int i = 0; i < N; i++)
{
    sem_destroy(&s[i]);
}

sem_destroy(&mutex);

printf("Todos os filosofos terminaram.\n");
return 0;
```

Hora de perguntar, dúvidas

Próxima aula

- Memória Principal e Memória Virtual: organização da memória, papel do sistema operacional no gerenciamento da memória, conceito de espaço de endereçamento virtual e motivação para o uso de memória virtual
- Paginação: estrutura de páginas e quadros, tabelas de páginas, tradução de endereços e papel da TLB no desempenho do sistema. Atividade prática: simulação da tradução de endereços virtuais para endereços físicos utilizando páginas e quadros.